

OPERATING SYSTEMS AND SYSTEM PROGRAMMING

PRACTICAL – WEEK 10

Name: S Svara Haasini

Roll No: 2520090170

Task 1: Inode, Hard Link and Symbolic Link Analysis

Question:

Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

Output:

```

cd Practical_Week10
haasini@SSH:~/OSSP_upload/Practical_Week10$ echo "OSSP inode experiment" > original.
txt
haasini@SSH:~/OSSP_upload/Practical_Week10$ ls -li original.txt
41660 -rw-r--r-- 1 haasini haasini 22 Sep 12 06:42 original.txt
haasini@SSH:~/OSSP_upload/Practical_Week10$ stat original.txt
  File: original.txt
  Size: 22          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 41660         Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/ haasini)   Gid: ( 1000/ haasini)
Access: 2026-09-12 06:42:52.109190626 +0000
Modify: 2026-09-12 06:42:52.109190626 +0000
Change: 2026-09-12 06:42:52.109190626 +0000
Birth: 2026-09-12 06:42:52.109190626 +0000
haasini@SSH:~/OSSP_upload/Practical_Week10$ ln original.txt hardlink.txt
haasini@SSH:~/OSSP_upload/Practical_Week10$ ln -s original.txt symlink.txt
haasini@SSH:~/OSSP_upload/Practical_Week10$ stat original.txt
stat hardlink.txt
stat symlink.txt
  File: original.txt
  Size: 22          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 41660         Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 1000/ haasini)   Gid: ( 1000/ haasini)
Access: 2026-09-12 06:42:52.109190626 +0000
Modify: 2026-09-12 06:42:52.109190626 +0000
Change: 2026-09-12 06:43:57.395816728 +0000
Birth: 2026-09-12 06:42:52.109190626 +0000
  File: hardlink.txt
  Size: 22          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 41660         Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 1000/ haasini)   Gid: ( 1000/ haasini)
Access: 2026-09-12 06:42:52.109190626 +0000
Modify: 2026-09-12 06:42:52.109190626 +0000
Change: 2026-09-12 06:43:57.395816728 +0000
Birth: 2026-09-12 06:42:52.109190626 +0000
  File: symlink.txt -> original.txt
  Size: 12          Blocks: 0          IO Block: 4096   symbolic link
Device: 8,48      Inode: 41663         Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/ haasini)   Gid: ( 1000/ haasini)
Access: 2026-09-12 06:44:03.322470950 +0000
Modify: 2026-09-12 06:44:03.322470950 +0000
Change: 2026-09-12 06:44:03.322470950 +0000
Birth: 2026-09-12 06:44:03.322470950 +0000
haasini@SSH:~/OSSP_upload/Practical_Week10$ find . -samefile original.txt
./hardlink.txt
./original.txt
haasini@SSH:~/OSSP_upload/Practical_Week10$

```

Observations:

1. The ls -li command displays inode numbers and file metadata for directory contents.
2. original.txt and hardlink.txt share inode number 41660, indicating they reference the same data.
3. Hard links create multiple directory entries pointing to the same inode and data block.
4. symlink.txt has its own separate inode, indicating it is a distinct file object.
5. Symbolic links store a path reference rather than pointing directly to an inode.
6. The stat command shows file statistics including inode number, block allocation, and permissions.
7. IO Block size (4096) is different from inode number and represents allocation unit size.
8. Hard links and the original file share the same data; changes affect all references.
9. Symbolic links are independent files that must be resolved to access the target.
10. The find command with -samefile identifies all hard links to the same underlying inode.

Task 2: Memory-Mapped File I/O Using mmap()

Question:

Write a program comparing performance of traditional read/write and memory-mapped file I/O. Use `mmap()` to map a file into memory and compare execution times for file copying.

Source Code:

```
GNU nano 7.2 P10_T2.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>
#include <time.h>

#define BUFFER_SIZE 4096

double elapsed(struct timespec start, struct timespec end)
{
    return (end.tv_sec - start.tv_sec) +
           (end.tv_nsec - start.tv_nsec) / 1000000000.0;
}

int main(void)
{
    struct stat st;
    struct timespec start, end;

    int src = open("input.dat", O_RDONLY);

    if (src == -1)
    {
        perror("open");
        return 1;
    }

    fstat(src, &st);

    size_t size = st.st_size;

    printf("File size: %zu bytes\n\n", size);

    /* Traditional read/write */
    int dst = open("rw_copy.dat",
                  O_WRONLY | O_CREAT | O_TRUNC, 0644);

    char buffer[BUFFER_SIZE];
    ssize_t n;

    clock_gettime(CLOCK_MONOTONIC, &start);

    while ((n = read(src, buffer, BUFFER_SIZE)) > 0)
        write(dst, buffer, n);

    clock_gettime(CLOCK_MONOTONIC, &end);

    printf("read/write time : %.6f seconds\n",
```

```
    write(dst, buffer, n);

    clock_gettime(CLOCK_MONOTONIC, &end);

    printf("read/write time : %.6f seconds\n",
        elapsed(start, end));

    close(dst);

    /* mmap version */
    lseek(src, 0, SEEK_SET);

    int mmap_dst = open("mmap_copy.dat",
        O_RDWR | O_CREAT | O_TRUNC, 0644);

    ftruncate(mmap_dst, size);

    char *src_map = mmap(NULL, size, PROT_READ,
        MAP_PRIVATE, src, 0);

    char *dst_map = mmap(NULL, size,
        PROT_READ | PROT_WRITE,
        MAP_SHARED, mmap_dst, 0);

    if (src_map == MAP_FAILED || dst_map == MAP_FAILED)
    {
        perror("mmap");
        return 1;
    }

    clock_gettime(CLOCK_MONOTONIC, &start);

    memcpy(dst_map, src_map, size);
    msync(dst_map, size, MS_SYNC);

    clock_gettime(CLOCK_MONOTONIC, &end);

    printf("mmap() time      : %.6f seconds\n",
        elapsed(start, end));

    munmap(src_map, size);
    munmap(dst_map, size);

    close(src);
    close(mmap_dst);

    printf("\nMemory-mapped and traditional copies completed.\n");

    return 0;
}
```

Output:

```
haasini@SSH:~/OSSP_upload/Practical_Week10$ nano P10_T2.c
haasini@SSH:~/OSSP_upload/Practical_Week10$ gcc -Wall -Wextra P10_T2.c
-o P10_T2
./P10_T2
File size: 10485760 bytes

read/write time : 0.068510 seconds
mmap() time      : 0.057963 seconds

Memory-mapped and traditional copies completed.
```

Observations:

1. fstat() retrieves file metadata including file size and permissions.
2. ftruncate() allocates file space to the specified size on the destination file.
3. Traditional read/write uses a buffer to read chunks from source and write to destination.
4. mmap() maps the entire file into the virtual address space for efficient access.
5. PROT_READ specifies read protection for memory-mapped region.
6. PROT_WRITE specifies write protection for memory-mapped region.
7. MAP_PRIVATE creates a private copy-on-write mapping of the file.
8. MAP_SHARED creates a shared mapping that reflects modifications to the actual file.
9. mmap() time includes mapping overhead but avoids repeated read/write system calls.
10. Performance comparison shows relative efficiency of both I/O approaches for different file sizes.